PLAN PRACY ZESPOŁU · WERSJA 1.0

Kasa biletowa

w wariancie Pociąg

Od rozkładu jazdy do kodu biletu. Trzy osoby, trzy karty i jedno wspólne API — tak, żeby każdy mógł zacząć od razu, a integracja była tylko formalnością.

A
ROZKŁAD I WAGON
Osoba 1 · Lider

→

B
REZERWACJA
Osoba 2

→

C
ZAKUP
Osoba 3

3	4	15 min	120
OSOBY	ENDPOINTY	REZERWACJA	MIEJSC W DEMO
każda z własną kartą	jeden wspólny kontrakt	2 min w testach	3 × 2 × 20

ZASADA NR 1
Każdy zaczyna od zaślepki. Integrujemy dopiero wtedy, gdy karta jest gotowa — w kolejności A → B → C.

W TYM DOKUMENCIE

01 **KONTRAKT**
Decyzje, model danych, API
s. 2–4

02 **TRZY KARTY**
Zadania osób A, B i C
s. 5–7

03 **SPRAWDZENIE**
Integracja, demo, ryzyka
s. 8–9

Trzy osoby. Jeden kontrakt. Zero czekania.

01 · PUNKT STARTOWY

Najpierw wspólny grunt

Etap I to kasa biletowa bez płatności online. Jedna rezerwacja to jedno miejsce na jednym kursie: klient wybiera miejsce, trzyma je 15 minut, a potem kupuje bilet z kodem. Po wygaśnięciu rezerwacji miejsce automatycznie wraca do puli.

CEL NA KONIEC ETAPU

Pełny przepływ bez płatności, logowania i miejsc pośrednich:

`lista kursów → mapa wagonu → formularz → licznik → zakup → bilet`

Cztery zasady, które trzymają zespół razem

01
Jeden kontrakt
Typy, format seatId, kody błędów i TTL mieszkają w packages/shared. Nie kopiujemy ich między aplikacjami.

02
Zaślepki na start
Każda karta ma lokalny mock. Osoba 2 nie czeka na API A, a Osoba 3 na API B.

03
Integracja A → B → C
Najpierw kliknięcie miejsca, potem rezerwacja, na końcu zakup. Zgodnie z tym, jak łączą się karty.

04
Serwer jest prawdą
Licznik w przeglądarce tylko pokazuje expiresAt. O ważności rezerwacji decyduje API.

Trzy zasady implementacji

01
Importuj wspólne typy
packages/shared to jedyne źródło modelu, TTL i etykiet.

02
Rezerwuj atomowo
Sprawdzenie miejsca i zapis w jednej operacji.

03
Zakup idempotentnie
Drugi request zwraca ten sam kod, nie nowy bilet.

Kto za co odpowiada

KARTA	OSOBA	API	EKRAN	EFEKT
A	Osoba 1 Lider	`GET /api/events` `GET /api/events/:id`	Lista kursów Mapa wagonu	Wolne miejsce prowadzi do formularza B.
B	Osoba 2	`POST /api/reservations`	Formularz Licznik	Miejsce zajęte na 15 min; licznik działa.
C	Osoba 3	`POST /api/reservations/:id/purchase`	Przycisk zakupu Bilet	Kod biletu i czytelne wygaśnięcie.

PIERWSZY CHECKPOINT

Lider publikuje kontrakt i szkielet repo w pierwszym bloku. Osoby 2 i 3 startują równolegle na zaślepkach, ale kodują wyłącznie przeciwko wspólnym typom.

Poza zakresem: płatności online · logowanie · miejsca pośrednie · anulowanie rezerwacji · aktualizacja mapy w czasie rzeczywistym.

Rekomendacja: TypeScript + pnpm workspaces; Node/Fastify lub Express; React + Vite + React Router; Vitest. Stack nie jest narzucony — kontrakt ma znaczenie, framework nie.

02 · KONTRAKT

Cztery decyzje, które zdejmują niepewność

PDF zostawia kilka szczegółów otwartych. Przyjmujemy je raz, zapisujemy w repo i traktujemy jak część API — także wtedy, gdy zmieni się implementacja.

DECYZJA 01

`takenSeats` = string[]

Zwracamy wyłącznie seatId. Lista zawiera miejsca kupione oraz miejsca z ważną rezerwacją. Bez duplikatów; wygasłe rezerwacje są pomijane.

DECYZJA 02

Odmowa zakupu = `409`

Gdy rezerwacja wygasła: HTTP 409 i { "error": "RESERVATION_EXPIRED" }. Ekran C pokazuje komunikat, a miejsce wraca do puli.

DECYZJA 03

`seatId = "wagon-miejsce"`

Oba ID są tekstem. W demo wagony 5 i 6 mają miejsca 1–20. Przykład "5-42" pokazuje tylko format — nie jest danym testowym.

DECYZJA 04

TTL z jednego źródła

RESERVATION_TTL_MINUTES: domyślnie 15, w testach 2. expiresAt to ISO UTC; ważność liczy wspólny isReservationActive().

Model danych

```
Event {
  id, relation, date,
  departureTime, arrivalTime,
  wagons: Wagon[]
}
EventSummary = Omit<Event, "wagons">
Wagon { id, number, seats: Seat[] }
Seat  { id, number, position }

position: "window" | "aisle"
```

```
Reservation {
  id, eventId, seatId,
  customerName, email,
  createdAt, expiresAt,
  status: "active"
        | "purchased"
        | "expired",
  code?: string
}
```

`seatId: "5-12" (string)` · `eventId: "ic-7310-2026-11-21" (string)` · `takenSeats: string[]`

Ważność rezerwacji — jeden mechanizm dla A, B i C

1

B zapisuje

expiresAt = createdAt + TTL; czas w ISO UTC.

2

A czyta

takenSeats = zakupy + aktywne rezerwacje.

3

C kupuje

now >= expiresAt → 409; miejsce wraca.

BEZ CRONA

Wygaszanie jest leniwe: każdy odczyt i zakup sprawdza czas. To samo źródło prawdy obsługuje mapę, formularz i zakup.

03 · API I PRZEPŁYW

Jeden kontrakt, cztery endpointy

Każdy endpoint ma jednego właściciela, ale wszyscy korzystają z tych samych typów i tego samego magazynu rezerwacji.

ENDPOINT	WEJŚCIE	SUKCES	ODMOWY	OWNER
GET /api/events	—	200 EventSummary[]	–	O1
GET /api/events/:id	—	200 { event, takenSeats }	404 EVENT_NOT_FOUND	O1
POST /api/reservations	{ eventId, seatId, customerName, email }	201 { reservationId, expiresAt }	400 VALIDATION_ERROR 404 EVENT_OR_SEAT_NOT_FOUND 409 SEAT_UNAVAILABLE	O2
POST /api/reservations/:id/purchase	—	200 { code }	404 RESERVATION_NOT_FOUND 409 RESERVATION_EXPIRED	O3

Co niesie kliknięcie, a co rezerwacja

PRZEJŚCIE	ADRES	DANE	ZASADA
A → B	/events/:eventId/reserve/:seatId	eventId, seatId; opcjonalnie snapshot eventu	B może pobrać event ponownie; URL działa po odświeżeniu.
B → licznik	/reservations/:reservationId	ReservationView w location.state + sessionStorage	Po odświeżeniu dane zostają; ID jest w URL.
C → bilet	/reservations/:id/ticket	{ code } + ReservationView	Zapis w sessionStorage; bez dodatkowego GET.

> **RESERVATIONVIEW — JEDEN OBIEKT MIĘDZY B I C**
> reservationId · expiresAt · event (summary) · seatId · customerName · email. Etykietę „Wagon 5, miejsce 12” liczymy z seatId przez formatSeatLabel().

Przykładowe odpowiedzi

SUKCES

```
GET /api/events/:id
200 { event, takenSeats: ["5-3", "5-7"] }

POST /api/reservations
201 { reservationId, expiresAt }

POST /api/reservations/:id/purchase
200 { code: "PAS-7K2M9Q" }
```

ODMOWA

```
GET /api/events/unknown
404 { error: "EVENT_NOT_FOUND" }

POST /api/reservations  (zajęte)
409 { error: "SEAT_UNAVAILABLE" }

POST /api/reservations/:id/purchase
409 { error: "RESERVATION_EXPIRED" }
```

Wszystkie błędy mają wspólny envelope { error: string }; status HTTP niesie znaczenie. Sukces nigdy nie zwraca error.

Jedno repo, trzy przestrzenie

apps/api
4 endpointy + store rezerwacji

apps/web
ekrany A, B i C

packages/shared
typy, TTL, fixtures, etykiety

A

OSOBA 1 / 3 · LIDER + KARTA A

Rozkład i wagon

`GET /api/events` · `GET /api/events/:id`

Ustawia wspólny grunt, a potem dowozi ekran, z którego klient wybiera kurs i wolne miejsce.

EFEKT	ZALEŻY OD	PRZEKAZUJE
Klik wolnego miejsca otwiera formularz B z poprawnym eventId i seatId.	Niczego — startuje pierwszy i publikuje kontrakt.	Kontrakt, fixtures, API i trasa do B.

Zadania

- ☐ **01 Zamknij kontrakt i szkielet repo**
  - packages/shared: typy, RESERVATION_TTL_MINUTES, isReservationActive, formatSeatLabel, fixtures.
  - CONTRACT.md: endpointy, błędy, trasy i ReservationView; 15-min review z O2 i O3.
- ☐ **02 Przygotuj dane demo i API A**
  - 3 kursy × 2 wagony (5 i 6) × 20 miejsc; numeracja 1–20, okno/korytarz z numeru.
  - GET /api/events i /:id; 404 dla nieznanego kursu; takenSeats z tego samego store'u co B.
- ☐ **03 Zbuduj listę kursów**
  - Relacja, data, odjazd, przyjazd; klik w kurs otwiera mapę wagonu.
  - Loading, empty i error; dane wyłącznie z API, bez hardkodowania w JSX.
- ☐ **04 Zbuduj mapę wagonu**
  - Przełącznik wagonów 5/6; siatka 5 rzędów × 4 miejsca; legenda okno/korytarz.
  - Zajęte wyszarzone i nieklikalne; wolne → /events/:eventId/reserve/:seatId.
  - Refetch po powrocie i na focus; ręczny przycisk odświeżania.
- ☐ **05 Spnij mapę z formularzem B**
  - Sprawdź, że aktywna rezerwacja B pojawia się w takenSeats po odświeżeniu.
  - Klik prowadzi do B z eventId, seatId i etykietą „Wagon 5, miejsce 12”.
- ☐ **06 Domknij jakość, ryzyka i demo**
  - Testy API A + test mapy; build bez błędów TS; README z uruchomieniem i TTL.
  - Wybierz 2–3 karty z listy zagrożeń; pokaż pełny przepływ na demo.
  - Lider: prowadź checkpointy integracji i pilnuj jednego źródła logiki.

GOTOWE, GDY...

- GET /api/events zwraca 3 kursy; /:id zwraca event + takenSeats, a nieznany id daje 404.
- Oba wagony mają po 20 miejsc; okno/korytarz są widoczne, zajęte miejsca zablokowane.
- Aktywna rezerwacja B blokuje miejsce; wygasła znika po odświeżeniu.
- Klik wolnego miejsca prowadzi do B z poprawnym eventId i seatId.
- Testy przechodzą, a README wystarcza do uruchomienia projektu.

Poza kartą A: jesteś właścicielem kontraktu i checkpointów integracji — nie implementujesz za innych.

B

OSOBA 2 / 3 · KARTA B

Rezerwacja

`POST /api/reservations` · licznik 15 minut

Zamienia wybrane miejsce w rezerwację na 15 minut i pokazuje klientowi, ile czasu zostało.

EFEKT	ZALEŻY OD	PRZEKAZUJE
Miejsce jest zajęte dla innych, a ekran licznika czeka na przycisk C.	Kontraktu i fixtures od O1 — na start lokalna zaślepka.	API rezerwacji, ReservationView i ekran licznika ze slotem dla C.

Zadania

- ☐ **01 Wystartuj na zaślepce**
  - Wspólne typy + fixtures; lokalny event i mock ReservationView.
  - Nie czekaj na ekran A ani API eventów — koduj przeciwko kontraktowi.
- ☐ **02 Zbuduj API rezerwacji**
  - POST /api/reservations → 201 { reservationId, expiresAt }.
  - Walidacja 400, brak eventu/miejsca 404, zajęte 409; zapis atomowy.
- ☐ **03 Podłącz wspólne wygasanie**
  - expiresAt = now + TTL; env domyślnie 15, w testach 2.
  - Aktywna rezerwacja w takenSeats A; po TTL miejsce wraca; bez crona.
- ☐ **04 Zbuduj formularz**
  - Trasa /events/:eventId/reserve/:seatId; etykieta z formatSeatLabel.
  - Imię, e-mail, walidacja, loading i błąd zajętego miejsca; sukces → ReservationView.
- ☐ **05 Zbuduj ekran licznika**
  - Trasa /reservations/:id; dane w sessionStorage; countdown z expiresAt serwera.
  - Po 0: „Rezerwacja wygasła…"; zostaw slot PurchasePanel dla C.
- ☐ **06 Przetestuj i przekaż C**
  - API: sukces, duplikat, 400/404, wygaśnięcie 2 min; UI: formularz + licznik.
  - Przekaż ReservationView, trasę licznika i mount point dla C.

GOTOWE, GDY...

- Rezerwacja jest widoczna w A; drugie zajęcie tego samego miejsca daje 409.
- Po 2 minutach testowego TTL miejsce wraca, a zakup jest niemożliwy.
- Formularz pokazuje dokładnie „Wagon 5, miejsce 12"; błędy są czytelne.
- Licznik liczy od expiresAt z serwera, działa po odświeżeniu, a 0 → wygasła.
- Slot na przycisk C jest gotowy; w kodzie B nie ma logiki zakupu.

Nie dodawaj przycisku „Kupuję bilet". To karta C — ty przygotowujesz dla niej miejsce i dane.

C

OSOBA 3 / 3 · KARTA C

Zakup

`POST /api/reservations/:id/purchase` · `{ code }`

Domyka przepływ: klient klika „Kupuję bilet”, dostaje kod, a wygasła rezerwacja nie kończy się biletem.

EFEKT	ZALEŻY OD	PRZEKAZUJE
Bilet z relacją, odjazdem, wagonem, miejscem i kodem; czytelne wygaśnięcie.	ReservationView i ekranu licznika od O2 — na start mock.	API zakupu, PurchasePanel i ekran biletu.

Zadania

- ☐ **01 Wystartuj na mocku**
  - Mock ReservationView ważny i wygasły; zbuduj PurchasePanel + ekran biletu.
  - Nie czekaj na API B — dane wejściowe są już w kontrakcie.
- ☐ **02 Zbuduj API zakupu**
  - Aktywna → unikalny kod PAS-XXXXXX, status purchased, 200 { code }.
  - Wygasła → 409 RESERVATION_EXPIRED; brak rezerwacji → 404; podwójny zakup = ten sam kod.
- ☐ **03 Dodaj przycisk na ekranie B**
  - PurchasePanel w slocie B; disabled przy wygaśnięciu i w trakcie requestu.
  - Sukces → bilet; 409 → komunikat i blokada zakupu.
- ☐ **04 Zbuduj ekran biletu**
  - /reservations/:id/ticket: relacja, data, odjazd, wagon, miejsce, kod.
  - Kod duży i wyraźny; powrót do rozkładu; dane w sessionStorage.
- ☐ **05 Obsłuż wygaśnięcie**
  - Licznik 0 lub 409 → „Rezerwacja wygasła — miejsce wróciło do puli”.
  - Brak biletu; miejsce widoczne jako wolne w A.
- ☐ **06 Przetestuj i spnij z B**
  - API: sukces, 404, wygasła, podwójny zakup; E2E rezerwacja → licznik → bilet.
  - Test z TTL=2; merge bez duplikowania licznika lub stanu rezerwacji.

GOTOWE, GDY...

- Zakup aktywnej rezerwacji zwraca 200 { code }, a bilet ma wszystkie wymagane dane.
- Wygasła rezerwacja daje 409 z RESERVATION_EXPIRED, komunikat i brak biletu.
- Podwójne kliknięcie nie tworzy dwóch kodów — drugi request zwraca ten sam.
- Przycisk jest osadzony w ekranie B; nie ma drugiego licznika ani drugiego modelu.
- E2E: happy path oraz ścieżka wygaśnięcia przechodzą z TTL=2 min.

Nie zmieniaj modelu rezerwacji ani logiki wygasania — korzystasz z tego samego packages/shared co B.

04 · INTEGRACJA

Spinamy całość — A, potem B, potem C

Integracja idzie dokładnie tak, jak łączą się karty. Każdy checkpoint ma konkretny dowód, że można iść dalej.

#	BRAMKA	KTO	DOWÓD
0	**Kontrakt i szkielet**	O1 + review O2/O3	shared + CONTRACT.md; każdy uruchamia repo.
1	**A na zaślepce**	O1	Mapa + klik do B; takenSeats puste.
2	**A + B**	O1 + O2	Aktywna rezerwacja w takenSeats; duplikat miejsca 409.
3	**B + C**	O2 + O3	Licznik → zakup → bilet; wygaśnięcie → komunikat.
4	**Całość**	wszyscy	Happy path + expiry + TTL 2 min na jednym demo.

Checklista demo end-to-end

- ☐ 3 kursy, każdy z 2 wagonami po 20 miejsc.
- ☐ Mapa pokazuje okno/korytarz; zajęte są wyszarzone.
- ☐ Klik wolnego miejsca → formularz z poprawną etykietą.
- ☐ Rezerwacja → licznik; drugi klient widzi miejsce jako zajęte.
- ☐ Zakup → bilet z kodem; kod jest widoczny.
- ☐ TTL=2 min → miejsce wraca, zakup daje 409 i komunikat.
- ☐ Duplikat miejsca 409; podwójny zakup = jeden kod.
- ☐ Refresh licznika i biletu nie gubi danych.

Demo w pięciu krokach

1	2	3	4	5
Wybierz kurs	**Wybierz miejsce**	**Zarezerwuj**	**Kupuję bilet**	**Pokaż bilet**
Lista A	Mapa A	Formularz B	Licznik B + C	Ekran C

Trzy przekazania, które muszą zadziałać

O1 → O2
`shared + fixtures + event API`
B dostaje event i miejsca bez własnego modelu.

O2 → O3
`ReservationView + slot`
C dostaje rezerwację i miejsce na CTA; bez drugiego GET.

O3 → O1
`purchase + 409`
A widzi miejsce wolne po wygaśnięciu; ten sam TTL.

DEFINICJA UKOŃCZENIA ETAPU I

Cała checklista jest odhaczona, TTL=2 min działa, podwójne rezerwacje są zablokowane, a README uruchamia projekt od zera.

05 · RYZYKA

Co może się rozjechać — i jak temu zapobiegamy

Największe ryzyko to nie brak funkcji, a dwa różne źródła prawdy o miejscu i czasie. Dlatego kontrakt i logika wygasania są wspólne.

ID	RYZYKO	ZABEZPIECZENIE	OWNER
R1	A i B inaczej rozumieją „zajęte”.	Wspólny isReservationActive; test: aktywna rezerwacja jest w takenSeats.	O1
R2	TTL albo zegar rozjeżdża się między API i UI.	Jedna zmienna + wspólny helper; countdown tylko z expiresAt; test TTL=2.	O2
R3	Dwa requesty zajmują to samo miejsce.	Atomowe create + unikalność (eventId, seatId) dla active/purchased.	O2
R4	Wygasła rezerwacja nadal blokuje miejsce.	Leniwe wygaszanie: takenSeats i purchase sprawdzają czas.	O1/O2/O3
R5	Mapa nie odświeża się po powrocie z formularza.	Refetch on focus/return + ręczny refresh; test po rezerwacji.	O1
R6	Konflikt na ekranie licznika między B i C.	B owns screen, C owns PurchasePanel; jeden slot i jeden PR.	O2/O3
R7	Refresh gubi dane biletu lub rezerwacji.	ID w URL + sessionStorage; bezpieczny fallback bez dodatkowego GET.	O2/O3

KARTY Z LISTY ZAGROŻEŃ

Lider wybiera 2–3 karty z oficjalnej listy prowadzącego, wpisuje ich numery i właścicieli do tego rejestru. Tabela powyżej zbiera ryzyka techniczne wynikające z planu.

Trzy decyzje, które muszą przetrwać

- takenSeats = zakupy + ważne rezerwacje, bez wygasłych.
- Odmowa zakupu to 409 z RESERVATION_EXPIRED; miejsce wraca automatycznie.
- Jedna definicja TTL i jedna funkcja isReservationActive dla A, B i C.

Zanim zaczniecie

- ☐ Kontrakt zaakceptowany przez O2 i O3; każdy wie, co dostaje.
- ☐ Każda osoba ma zaślepkę i pierwszy task.
- ☐ TTL=2 min w testach, 15 min domyślnie; wspólny helper.
- ☐ 2–3 karty ryzyka z listy prowadzącego są w rejestrze.

Gotowe do wysłania zespołowi, gdy lider uzupełni karty ryzyka i wszyscy zaakceptują kontrakt na kick-offie.